

ATIVIDADE 2 - Criação de Processos

SISTEMAS OPERACIONAIS

Aluna: Mirelle Casimiro Silvino

Resolução dos exercícios — fork() e execve()

Exercício 1 — fork.c

1. Código comentado

```
1  #include <unistd.h>          // fork(), getpid(), getppid(), sleep()
2  #include <stdio.h>           // printf(), perror()
3  #include <stdlib.h>          // exit()
4  #include <sys/types.h>       // tipos de dados do sistema (pid_t)
5  #include <sys/wait.h>        // wait()
6
7  int main ()
8  {
9      int retval ;             // guarda o valor devolvido por fork()
10
11     // Executado UMA única vez, pelo processo original (que será o pai)
12     printf ("Ola, sou o processo %5d\n", getpid()) ;
13
14     // Cria um novo processo, cópia do atual. A partir daqui existem DOIS
15     // processos executando este mesmo código, na instrução seguinte ao
    fork:
16     // no pai : retval = PID do filho (> 0)
17     // no filho : retval = 0
```

```

18  // em erro : retval = -1 (nenhum filho é criado)
19  retval = fork () ;
20
21  // Executado pelos DOIS processos (a ordem entre eles não é
    determinística)
22  printf ("[retval: %5d] sou %5d, filho de %5d\n", retval, getpid(), getppid()) ;
23
24  if ( retval < 0 )    // erro no fork()
25  {
26      perror ("Erro") ; // imprime a mensagem de erro associada a errno
27      exit (1) ;        // termina com código de erro
28  }
29  else
30      if ( retval > 0 ) // sou o processo pai
31          wait (0) ;    // bloqueia até que o filho termine
32      else              // sou o processo filho
33          sleep (5) ;    // suspende-se por 5 segundos
34
35  // Executado pelos dois: o filho após 5 s; o pai após o término do filho
36  printf ("Tchau de %5d!\n", getpid()) ;
37  exit (0) ;            // término normal
38  }

```

O programa começa com um único processo, que imprime "Ola, sou o processo ..." uma só vez. Em seguida chama fork(), que duplica esse processo: a partir dali existem dois, o pai e o filho, e os dois continuam executando a partir da linha logo depois do fork(). O que muda entre eles é o valor de retval, que no pai é o PID do filho e no filho é 0, além do PID de cada um.

Como o segundo printf vem depois do fork(), os dois processos o executam. No pai, getppid() devolve o PID do shell que lançou o programa, e no filho devolve o PID do próprio pai. A ordem em que essas duas linhas aparecem depende do escalonador, então pode variar de uma execução para outra.

Depois disso, o if/else separa os caminhos. O pai (retval > 0) chama wait() e fica bloqueado esperando o filho terminar, enquanto o filho (retval == 0) chama sleep(5) e fica suspenso por 5 segundos. Passado esse tempo, o filho imprime seu "Tchau" e termina, e só então o wait() do pai retorna e ele imprime o dele. Por isso o filho sempre termina antes do pai.

Exemplo de saída (PIDs ilustrativos: shell = 3000, pai = 4242, filho = 4243):

Ola, sou o processo 4242

[retval: 4243] sou 4242, filho de 3000

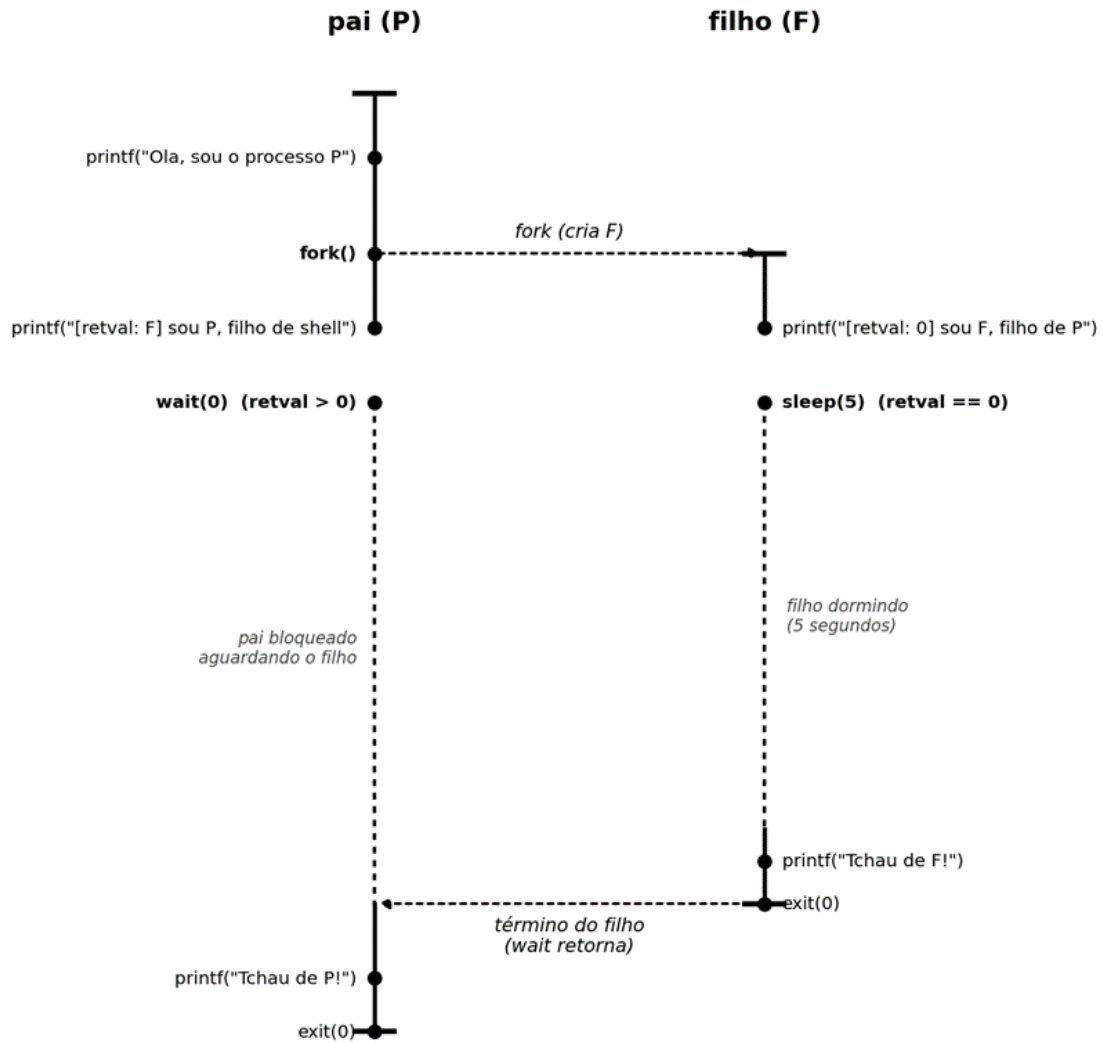
[retval: 0] sou 4243, filho de 4242

(... 5 segundos de pausa ...)

Tchau de 4243!

Tchau de 4242!

2 - Diagrama de tempo

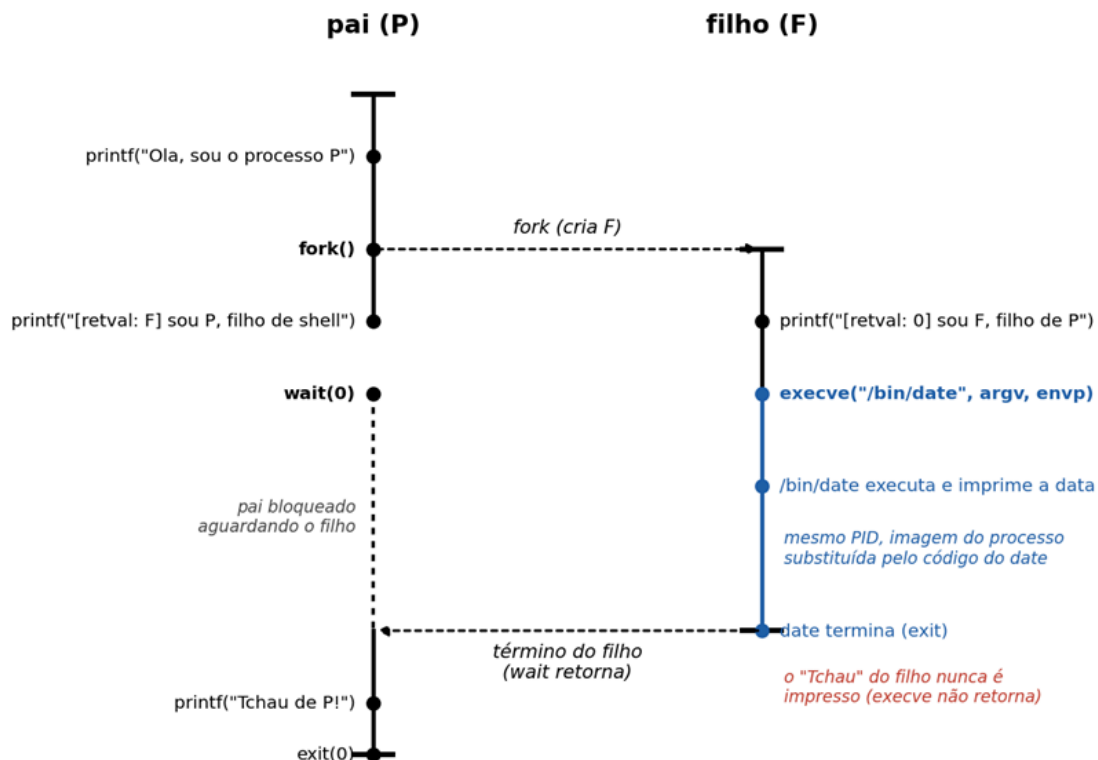


Exercício 2 — fork-execve.c

1 - Código comentado:

```
#include <unistd.h> // fork(), execve(), getpid(), getppid()
#include <stdio.h> // printf(), perror()
#include <stdlib.h> // exit()
#include <sys/types.h>
#include <sys/wait.h> // wait()
// argv = argumentos da linha de comando; envp = variáveis de ambiente
int main (int argc, char *argv[], char *envp[])
{
    int retval ;
    // Impresso uma vez, pelo processo original
    printf ("Ola, sou o processo %5d\n", getpid()) ;
    // Cria o processo filho (cópia do pai)
    retval = fork () ;
    // Impresso pelos dois processos
    printf ("[retval: %5d] sou %5d, filho de %5d\n", retval, getpid(), getppid()) ;
    if ( retval < 0 ) // erro no fork()
    {
        perror ("Erro: ") ;
        exit (1) ;
    }
    else
    if ( retval > 0 ) // sou o processo pai
        wait (0) ; // espera o término do filho
    else // sou o processo filho
    {
        // Substitui a imagem do processo (código, dados, pilha) pelo programa
        // /bin/date. O PID NÃO muda. Se der certo, execve() nunca retorna.
        execve ("/bin/date", argv, envp) ;
        // Só chega aqui se execve() FALHAR (ex.: arquivo inexistente)
        perror ("Erro") ;
    }
    // Pai: após o filho terminar. Filho: só se o execve tiver falhado.
    printf ("Tchau de %5d!\n", getpid()) ;
    exit (0) ;
}
```

2 - Diagrama de tempo



3 - O que ocorre se o programa a ser executado por `execve` não existir? Desenhe o diagrama de tempo que retrata essa situação:

Se o programa passado ao `execve()` não existir, a chamada falha. Não há nada para substituir a imagem do processo, então nada é trocado e o `execve()` retorna -1, com `errno = ENOENT` ("No such file or directory"). Com isso, o filho continua executando o código do programa original.

A linha seguinte, `perror("Erro")`, é executada e imprime algo como "Erro: No such file or directory". Depois, o fluxo sai do bloco `else` e chega ao `printf("Tchau de ...")` e ao `exit(0)`, de modo que o filho imprime o seu próprio "Tchau" e termina normalmente. O pai, que estava bloqueado no `wait()`, é então liberado, imprime o seu "Tchau" e termina. O resultado é que aparecem dois "Tchau", um do filho e outro do pai, e nenhuma data é impressa. Em programas reais, o filho deveria chamar `exit(1)` logo após o `perror`, para não continuar executando código que era destinado ao pai.

Exemplo de saída (trocando o caminho por um arquivo inexistente, como "/bin/dtex"):

Ola, sou o processo 4242

[retval: 4243] sou 4242, filho de 3000

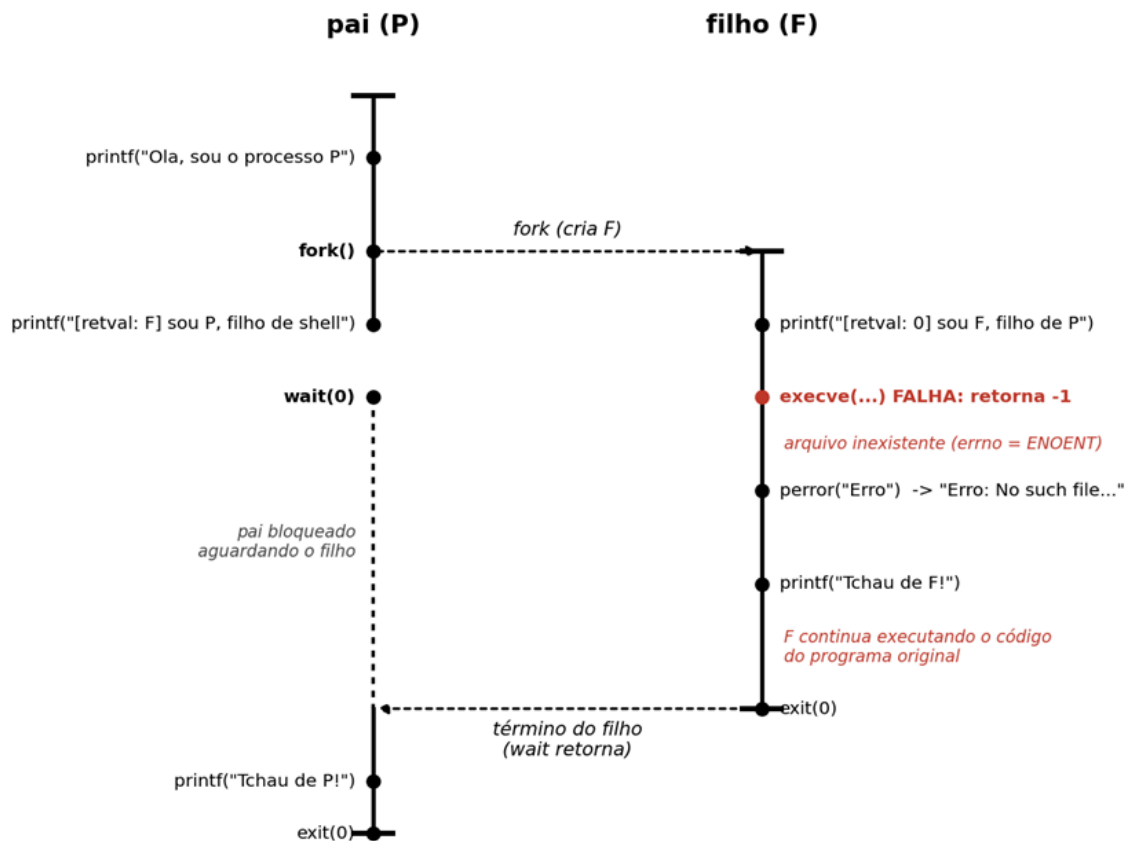
[retval: 0] sou 4243, filho de 4242

Erro: No such file or directory

Tchau de 4243!

Tchau de 4242!

Diagrama de tempo:



Exercício 3 — fork-print.c

1- Código comentado:

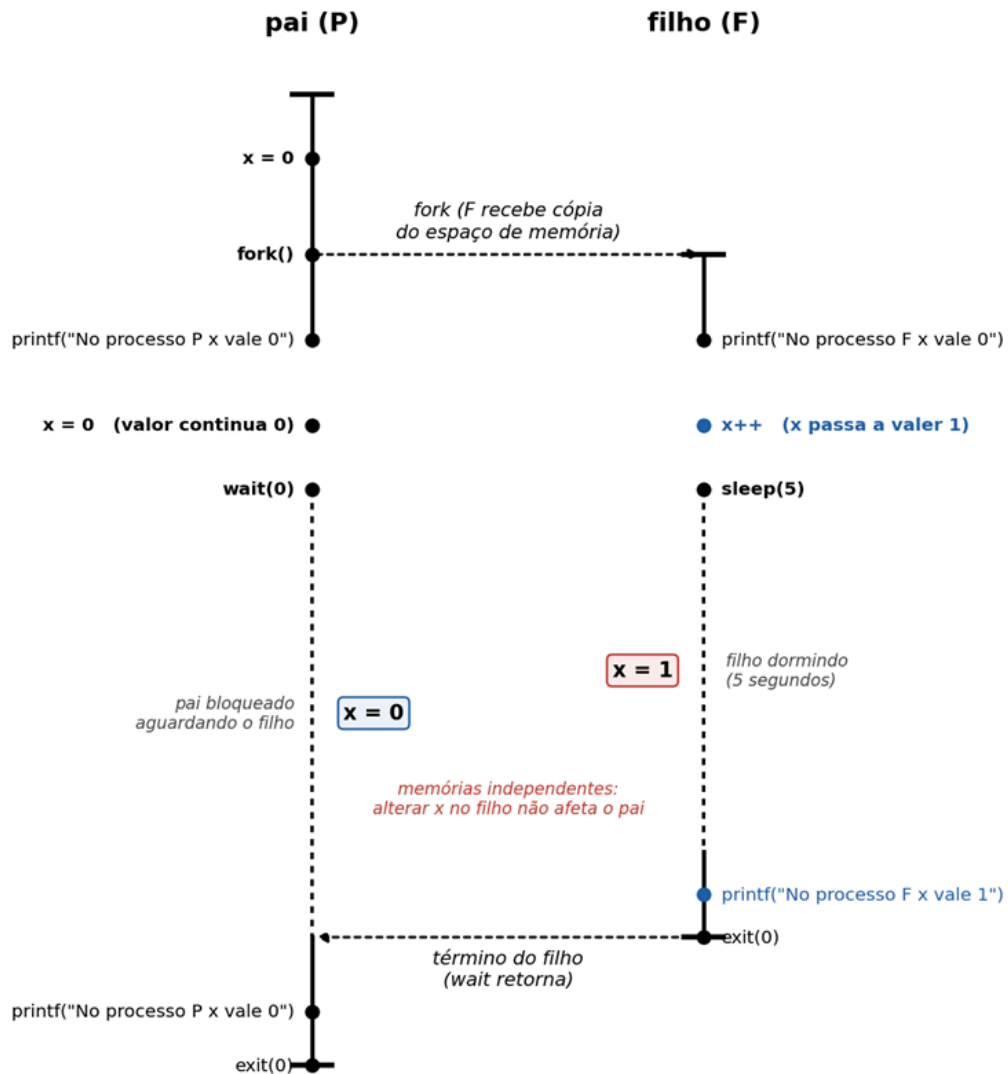
O programa começa incluindo os cabeçalhos necessários: `unistd.h`, que fornece `fork()`, `getpid()` e `sleep()`; `stdio.h`, para `printf()` e `perror()`; `stdlib.h`, para `exit()`; `sys/types.h`, para os tipos de dados do sistema; e `sys/wait.h`, para `wait()`. No `main`, são declaradas duas variáveis inteiras: `retval`, que guardará o valor devolvido pelo `fork()`, e `x`, a variável de teste. Em seguida, `x` recebe o valor 0, ainda antes do `fork`.

Depois vem a chamada `retval = fork()`, que cria o filho. Ele recebe uma cópia do espaço de memória do pai, incluindo a variável `x`, que vale 0 nos dois processos neste ponto. Logo em seguida, os dois processos executam o mesmo `printf` e imprimem "No processo (PID) x vale 0".

O `if/else` então separa os caminhos. Se `retval` for menor que 0, o `fork()` falhou: o programa imprime a mensagem de erro com `perror()` e termina com `exit(1)`. Se `retval` for maior que 0, é o processo pai, que atribui 0 ao seu próprio `x` (sem efeito prático, já que ele já valia 0) e chama `wait(0)` para esperar o filho terminar. Se `retval` for igual a 0, é o processo filho, que incrementa o seu próprio `x`, passando a valer 1, e depois chama `sleep(5)` para dormir por 5 segundos.

Por fim, os dois processos executam o último `printf`: o filho imprime `x = 1` e o pai imprime `x = 0`, porque as memórias são independentes. Depois disso, cada um termina com `exit(0)`.

2 - Diagrama de tempo:



EVOLUÇÃO DE X:

Momento	x no pai (P)	x no filho (F)
Antes do <code>fork()</code>	0	(não existe)
Logo após o <code>fork()</code> / 1º <code>printf</code>	0	0
Após o <code>if/else</code>	0 ($x = 0$)	1 ($x++$)
2º <code>printf</code> (final)	0	1